

CA3 Reflective Document: AI for Games

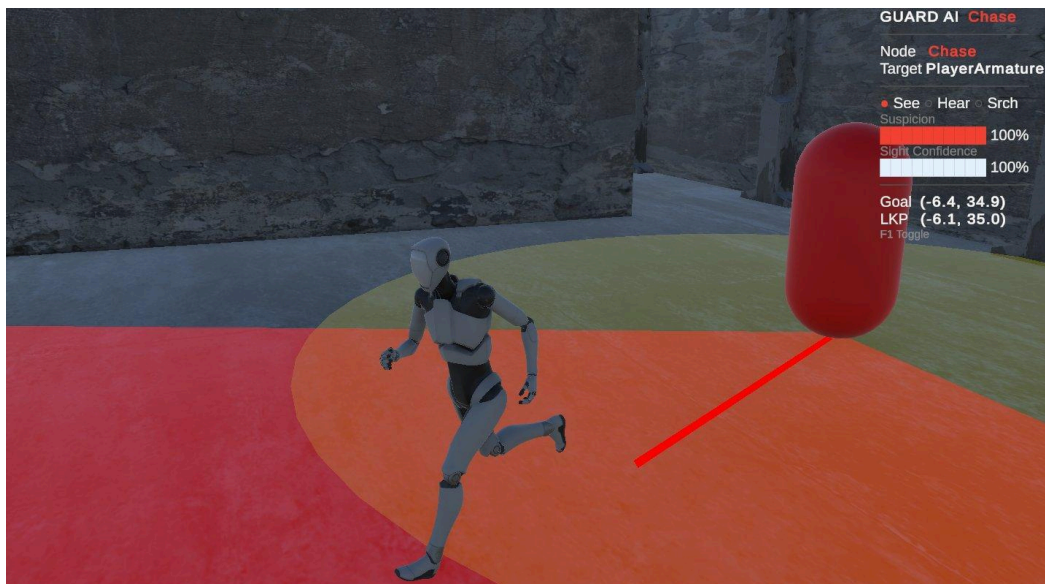
Name: Wolfgang Romanowski

Student Number: 20101931

Repository: <https://github.com/Wolfgang-Romanowski/GameDev-Unity-CA-Sem2>

Branch: feature/ca3-ai

Scene: Assets/Scenes/CA3.unity



Guard in Chase state. Debug overlay shows active BT node, perception bars, blackboard values, and goal and LKP coordinates. Cone, path line, and overlay text are all colour-coded to the active state.

Section 1: Integrated technical account

This document is the natural evolution of the technical material developed across CA1 and the critical discussion submitted for CA2, revised to reflect the integrated state of the AI vertical slice at CA3 submission. CA1 established the navigation layer and a four-state FSM. CA2 introduced a hand-rolled Behaviour Tree, a Blackboard, a SuspicionSystem accumulator, and an angle-cone vision check, CA3 closes the loop and makes the whole chain observable in real time through a debug overlay rebuilt from scratch.

System overview:

The CA3 AI vertical slice is a single-guard stealth encounter in which the player must reach a goal zone without being detected. The guard NPC runs three layers communicating through a shared Blackboard. Perception is GuardSensor (line-of-sight raycasts plus velocity-gated hearing) and SuspicionSystem (a 0 to 1 awareness value that decays when the player is not perceived), with a new SightConfidence accumulator that rises while LOS is held and decays when lost. Decision-making is GuardBT, a Behaviour Tree composed of a four-branch Selector (Chase, Investigate, Search, Patrol) with cooldown, timeout, and conditional-abort decorators. Action is GuardMovement (a NavMeshAgent wrapper) and GuardVisionCone (a dynamic cone mesh and hearing ring snapped to the floor).

Data flows perception, then Blackboard, then BT branch, then navigation. That is the spine of every design decision below.

Carry-forward from CA1 and CA2:

The navigation layer is inherited from CA1 with no architectural changes and the sliding-door dynamic obstacle is retained as the navigation complication and still triggers carving correctly. The BT framework (BTNode, BTComposite, LeafPatrol), SuspicionSystem, and four-state FSM (GuardAI) are inherited from CA2 with the original files left in Assets/Scripts/CA2/ exactly as marked. The Blackboard and vision-cone visualisation were carried forward conceptually from CA2 but live in Assets/Scripts/CA3_AI/ alongside the other CA3-edited code.

New work for CA3:

The CA2 marker awarded 91/100 and flagged two specific gaps (the CooldownDecorator on a child returning Success immediately, and LeafSearch.hasGoal not resetting on re-entry). Both are addressed in CA3, but the more substantive work was layering perception and decision-making so the encounter stays readable, fair, and debuggable across longer play sessions.

Sight Confidence is a 0 to 1 smoothed value in GuardSensor that rises at 1.5 per second while LOS is held and decays at 1.0 per second otherwise. This added hysteresis to chase logic: confidence above 0.7 enters chase, confidence above 0.3 keeps it running, so a single-frame LOS dropout no longer terminates the chase when the player briefly slips behind cover.

Velocity-gated hearing means hearing only registers if the player's speed exceeds `hearingSpeedThreshold` (1.5 m/s by default). Sneaking or standing still produces no suspicion at all, letting speed and positioning be tuned independently.

`LeafInvestigate` was hardened against the CA2 walk-to-LKP-and-bail by adding a `pathWaitDeadline` grace window of 1 second that tolerates `NavMesh` path resolution latency before declaring failure, and a 2-second linger phase at the LKP during which `movement.StartLookAround()` triggers in-place rotation so `Investigate` is visibly distinct from `Search`.

`LastKnownPosition` is now sampled through `NavMesh.SamplePosition` before being written to the Blackboard, eliminating a class of bugs where the player's transform Y sat just below the mesh and the agent silently returned a `PathPartial` status.

The vision-cone mesh now uses cached buffers reallocated only when `rayCount` changes, replacing per-frame allocations. Profiler evidence in Section 2.

`DebugOverlay` was rewritten to render active BT node, FSM state, both perception bars, goal coordinates, and LKP live each frame. The vision cone and hearing ring snap to the floor Y via a downward raycast so the visualisation hugs the ground regardless of the guard's pivot. F1 toggles overlay visibility for clean recording.

The four BT branches map one-to-one onto the CA1 FSM states and in CA3 the BT replaces the FSM as the high-level state layer rather than running alongside it. Layering a separate FSM on top of a Selector-based BT would have produced duplicated state and contradictory transitions, so the FSM enum is retained (`GuardAI.CurrentState`) purely as a colour-coding tag for the debug overlay and path line, not as an authority on transitions.

BT structure and decision flow:

The tree is a flat Selector with four priority-ordered branches:

Root: Selector

Chase (ConditionalAbort: `SightConfidence > 0.7 OR Suspicion >= 0.8 && !IsSearching`)

Investigate (ConditionalAbort outside Cooldown: `!IsSearching && Suspicion >= 0.3`)

Search (Sequence: `IsSearching` condition + `TimeoutDecorator(LeafSearch, 10s)`)

Patrol (default fallback)

The condition-outside-cooldown wrap on `Investigate` was a deliberate fix. With the cooldown on the outside, a below-threshold tick would arm the cooldown timer and starve the branch for 5 seconds. With the condition on the outside, the cooldown only ticks when the condition is true and only arms when the leaf actually ran. The same fix pattern was applied to the CA2 `CooldownDecorator` behaviour that the marker flagged.

The tree is ticked at 10 Hz via a coroutine (`btTickRate = 0.1f`) rather than every frame. The sensor runs each frame because perception responsiveness is more important than its cost; the

BT does not need 60 Hz decision-making. This is the most concrete tick-rate justification I can offer for the rubric's tick-rate-management criterion.

Distinct states evidenced:

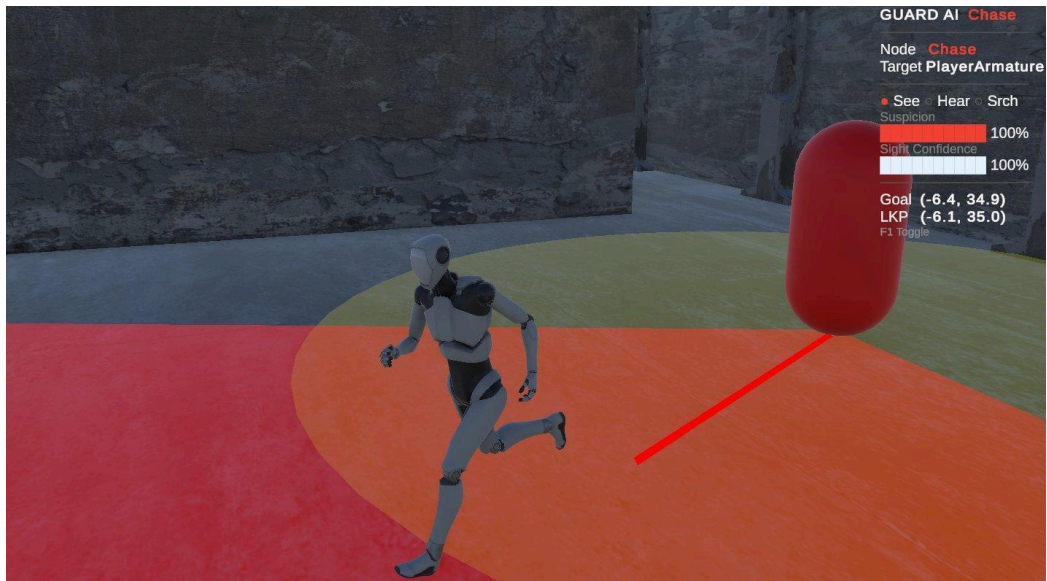
Four state screenshots are committed at Docs/Screenshots/CA3_AI_Screenshots/. Each was taken with the debug overlay active so the active BT node, blackboard values, and perception visualisation are all visible in the same frame.



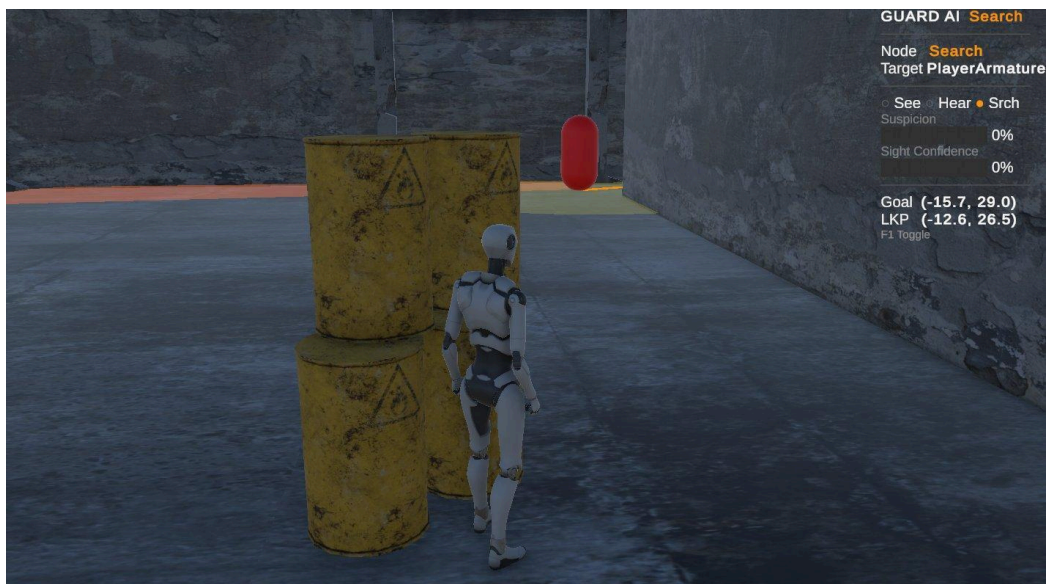
Patrol: waypoint cycling, suspicion 0%, sight confidence 0%, green cone, green path line.



Investigate: walking to LKP at patrol speed, suspicion in the 30 to 80% band, yellow cone, yellow path line. Sight Confidence remains at 0% because the player is behind cover.



Chase: direct pursuit, Suspicion and Sight Confidence both at 100%, red cone, red path line, See indicator lit.



Search: random NavMesh wander near LKP, IsSearching flag set, orange cone, Srch indicator lit. Suspicion has already decayed.

Section 2: Performance and robustness evaluation

What was profiled:

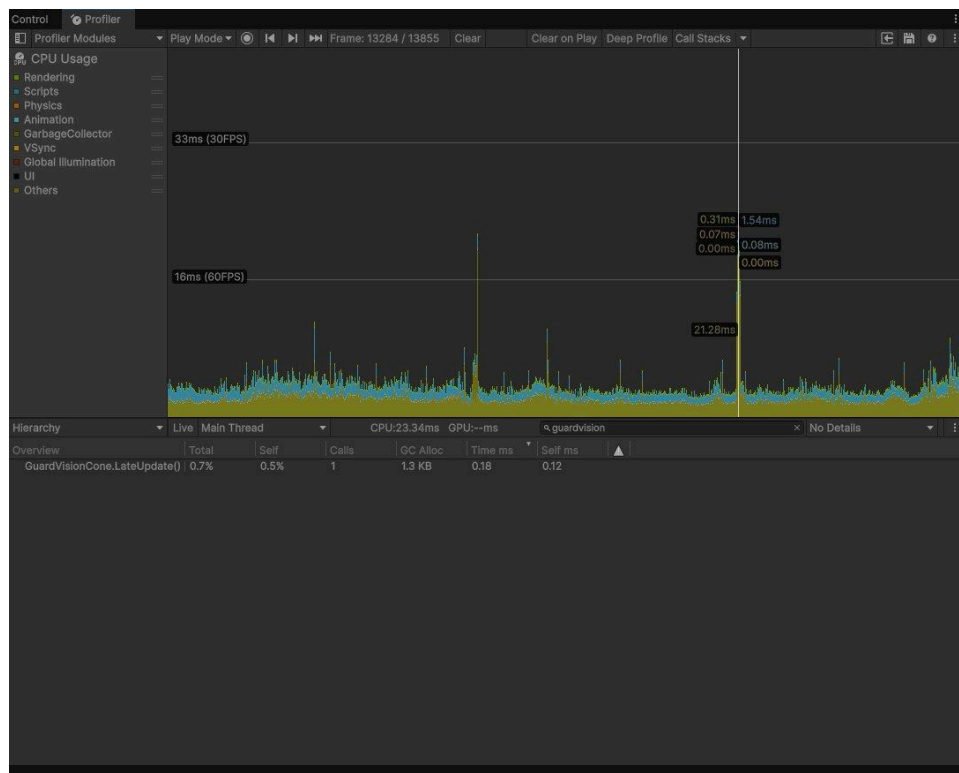
The Unity Profiler was run in Play Mode with the player active and the guard NPC cycling through Patrol, Investigate, Chase, and Search. The Hierarchy view was filtered to 'guardvision' to isolate the cone-rendering cost path. Three captures are committed to Docs/ProfilerCaptures/AI_Profiler/.

Most significant cost:

The headline finding was GuardVisionCone.DrawVisionCone(), called every LateUpdate. With rayCount set to 80 the original implementation allocated:

```
Vector3[] vertices = new Vector3[rayCount + 2]; // 82 elements
int[] triangles = new int[rayCount * 3]; // 240 elements
```

at 60 or more frames per second. The before capture shows GuardVisionCone.LateUpdate() consuming 23.34 ms CPU time with 1.3 KB of GC allocation per call, and the frame-time graph shows the dense allocation stripe that drives those periodic spikes. The 240 raycasts per frame (rayCount multiplied by HeightSteps, with HeightSteps set to 3 so the cone clips correctly against walls of varying height) are a deliberate quality-cost trade-off and were left as is.



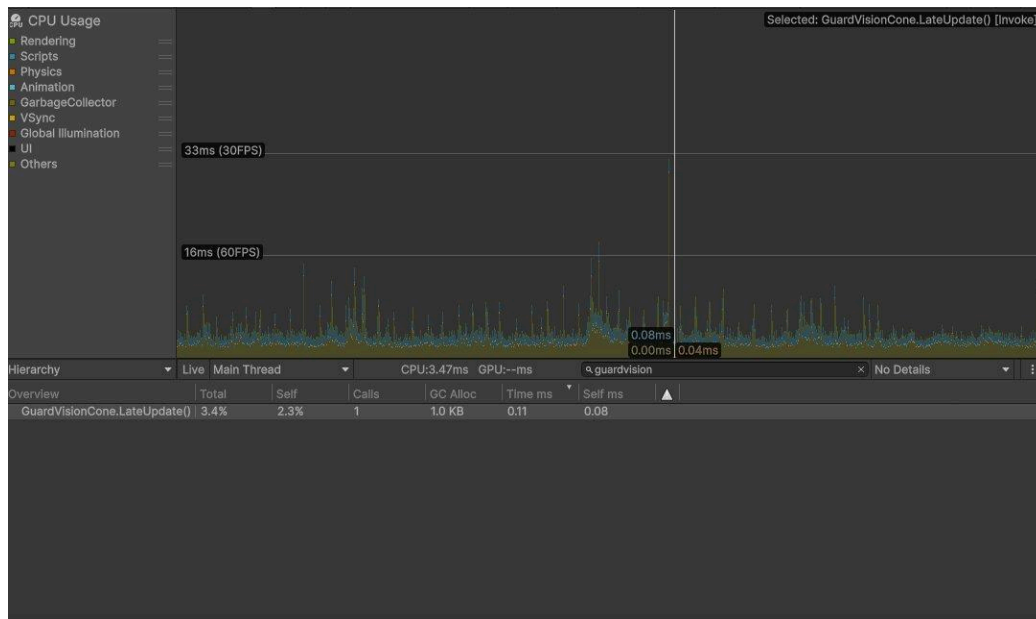
Before: GuardVisionCone.LateUpdate() consuming 23.34 ms with 1.3 KB GC allocation per frame. The yellow allocation stripe is visible across the timeline.

The fix:

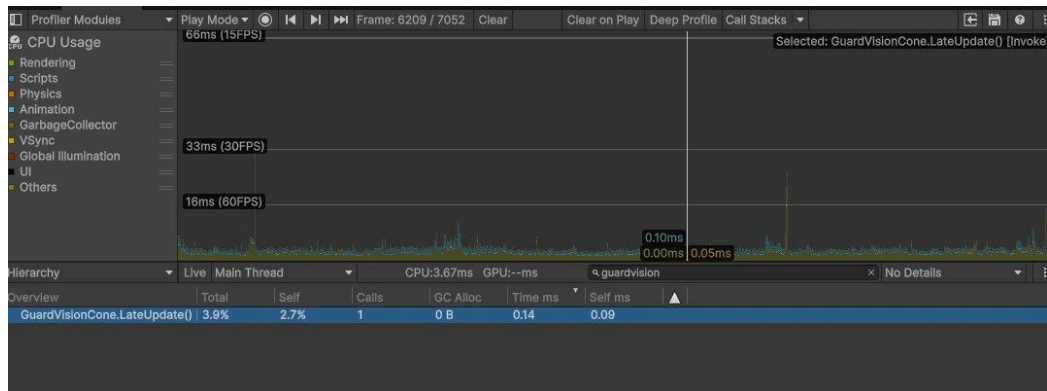
Both arrays became fields, reallocated only when rayCount actually changes:

```
if (coneVertices == null || lastRayCount != rayCount)
{
    coneVertices = new Vector3[rayCount + 2];
    coneTriangles = new int[rayCount * 3];
    lastRayCount = rayCount;
}
```

The after capture shows CPU time on the same method down to 3.47 ms with GC allocation reduced to 1.0 KB, and the final submission-state capture shows the GC column reading 0 B and frame time settled around 3.67 ms. The cone is still one of the more expensive components in the AI stack because the raycasts are still there, but it is no longer the GC hotspot.



After: CPU time on GuardVisionCone.LateUpdate() down to 3.47 ms, frame time stabilised under the 60 fps line.



Final submission-state capture: GC allocation column reads 0 B on the filtered row. The frame-time graph is flat with isolated spikes from unrelated systems.

Tick-rate management:

Three explicit choices, each justified in code comments or Tooltip attributes. The BT runs at 10 Hz via coroutine because decision-making does not need frame-rate granularity. The sensor runs every frame because perception responsiveness matters and the LOS raycast is a single short ray. The vision-cone mesh rebuild runs every frame because the cone geometry is rotation-dependent and any throttling produces visible lag in the overlay.

Edge cases resolved during development:

Three are worth recording. The first was LKP off-NavMesh. Investigate was failing instantly because `agent.SetDestination(player.position)` accepted a position fractionally below the mesh and `agent.pathStatus` returned `PathPartial`. Resolved with `NavMesh.SamplePosition` in `GuardSensor.SnapToNavMesh` so the Blackboard always holds an on-mesh LKP. The second was path resolution latency. Even after the snap fix, `IsValidPath` could return false on the BT tick immediately after `SetDestination` while Unity was still computing the path. Resolved with a `pathWaitDeadline` field in `LeafInvestigate` giving the navmesh up to 1 second to produce a complete path before declaring failure. The third was Investigate gate starvation, described in Section 1: inverting the decorator wrap order so the condition is checked first and only ticks the cooldown when the condition is true.

All three are subtle, and none of them was a wrong-algorithm problem. They were boundary conditions between Unity's NavMesh subsystem, Fusion-style tick discipline, and my own decorator composition. The honest take is that the BT framework I built in CA2 was correct on the happy path, and CA3 was about hardening it for the unhappy paths.

Section 3: AI in modern engines and beyond

Unity 6 versus Unreal Engine 5: AI tooling and architecture:

Game AI is a distinct discipline within games development, with its own techniques, tooling, and design challenges that do not map cleanly onto general AI or machine learning practice (Thompson, 2024). The two dominant engines for the practising indie or AAA developer, Unity and Unreal, make fundamentally different choices about how much of that discipline is built into the engine itself.

Unity 6 provides AI as a collection of optional packages rather than a single integrated stack. NavMesh baking, NavMeshAgent, and NavMeshObstacle (with carving) come from the AI Navigation package. Higher-level decision-making is not provided in the engine itself: until 2024, Behaviour Tree authoring relied on either third-party assets or, as in this submission, a custom implementation. Unity's own native answer is the Behavior package, which ships a graph-based authoring tool with action, modifier, sequencing, and join nodes plus a built-in Blackboard (Unity Technologies, 2024), addressing what was previously a real gap in the Unity ecosystem.

Unreal Engine 5 ships AI as an integrated first-class system. The standard stack is an AAIController driving a UBehaviorTree asset that references a UBlackboardData asset for shared state, with composite nodes (Selector, Sequence, Simple Parallel), decorators, and UAIPerceptionComponent for sight, hearing, and damage senses all provided out of the box. The Environment Query System (EQS) provides spatial reasoning of the kind 'find the best location that has line of sight to the player to attack' through a query language built from Generators, Contexts, and Tests, invocable directly from a BT node (Epic Games, 2024a). StateTree, introduced more recently, sits alongside BTs for state-heavy logic where BTs become unwieldy: as the Unreal Fest Gold Coast 2024 talk notes, BTs struggle with scripted sequences, multi-responsibility interruptions, and modular composition (Epic Games, 2024b).

Two architectural differences are worth naming. The first is the authoring model: Unreal's BT is a UAsset edited in a visual graph and Unity's current native equivalent is the Behavior graph, but Unity's package is much younger and its tooling is correspondingly less mature. The second, more significant, is spatial reasoning. EQS is closer to a query language for game-world geometry than a BT node, and its absence in Unity is the most consequential gap when comparing the two engines for AI work. In my own implementation, the random-goal-near-LKP behaviour in LeafSearch is a hand-rolled stand-in for what would be a one-line EQS query in Unreal. The trade-off, candidly, is that Unity's approach forces you to understand what you are doing. building LeafSearch by hand taught me more about pathfinding constraints than any black-box call would have.

Game AI techniques in serious games, military and industrial training:

Game AI techniques have transferred extensively into serious games for military and industrial training. The transferable elements are well-documented: NavMesh-driven crowd movement, behaviour trees for scripted enemy or trainee response, line-of-sight perception, and

parameterised difficulty scaling. The constraints that distinguish serious-games contexts from commercial games are equally well-documented and substantively different.

Evensen, Stien and Bentsen (2018) document the Norwegian Defence Research Establishment's (FFI) use of Bohemia Interactive Simulations' VBS Control framework to model semi-automated forces for entity-level constructive simulation of battalion-to-brigade operations. Their FFI report 18/01651, a NATO Modelling and Simulation Group symposium contribution, remains the clearest published account of BTs in military training. They retain the patterns this submission uses (Selector and Sequence composites, conditional decorators, leaf-level action nodes) but their priorities diverge: the BT is judged on whether instructors can recognise correct doctrine in the agent's behaviour, and trainee modelling plus post-session after-action review are first-class features. VBS Control's authoring tool ships with live debugging baked in for exactly that reason.

Hannaford (2018) makes the equivalent argument for industrial and medical training. His paper proposes BTs as a formal representation for medical procedures, arguing that the human-readability and composability that made BTs successful in games are exactly the properties that make them usable for procedural training where a clinical decision tree must be auditable and modifiable by domain experts who are not programmers. The same composite, decorator, and leaf structure I built for this submission would, in a clinical training simulator, be inspected and certified rather than tuned for fairness.

Three constraints distinguish these contexts from commercial games AI. Fidelity: trainees must learn behaviour that transfers to real-world performance, so the AI cannot be tuned for fun but must be tuned for accuracy against published doctrine or clinical procedure. Auditability: serious-games systems instrument the decision trace as a primary feature for instructor review, where the equivalent commercial debug overlay is a developer aid stripped from the shipping build. Regulatory constraint: industrial simulators used for nuclear, aviation, or surgical training operate under certification regimes where AI mispredictions could shape unsafe habits.

The connection back to my own work is concrete. The debug overlay I built for this submission is a developer's tool. In a serious-games context, the same overlay (node history, blackboard values, perception bars) is the foundation of the instructor's after-action review screen. What I built to help me find bugs is structurally the same artefact that, in an adjacent industry, is the product.

References:

Epic Games (2024a) *Environment Query System in Unreal Engine*.

Available at:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/environment-query-system-in-unreal-engine>

[Accessed 13 May 2026].

Epic Games (2024b) *Exploring the New State Tree for AI* [video]. Unreal Fest Gold Coast 2024.

Available at:

<https://www.youtube.com/watch?v=U8CkmOJg93M>

[Accessed 13 May 2026]

Evensen, P.-I., Stien, H. and Bentsen, D.H. (2018) *Modelling Human Behaviour using Behaviour Trees*. FFI Report 18/01651. Norwegian Defence Research Establishment. ISBN 978-82-464-3102-4.

Available at:

<https://www.ffi.no/en/publications-archive/modelling-human-behaviour-using-behaviour-trees>

[Accessed 13 May 2026].

Hannaford, B. (2018) *Behavior Trees as a Representation for Medical Procedures*.

arXiv:1801.07864. Biorobotics Laboratory, University of Washington.

Available at:

<https://arxiv.org/abs/1801.07864>

[Accessed 13 May 2026].

Thompson, T. (2024) *Why is It Difficult to Make Good AI for Games? AI 101* [video]. AI and Games.

Available at:

<https://www.youtube.com/watch?v=qCkqpRnk1oU>

[Accessed 13 May 2026]

Unity Technologies (2024) *Behavior Package Documentation, version 1.0*.

Available at:

<https://docs.unity3d.com/Packages/com.unity.behavior@1.0/manual/index.html>

[Accessed 13 May 2026]

Section 4: Carry-forward and professional reflection

One technical improvement to make:

The single improvement I would prioritise is replacing the distance-based hearing model with a noise-event system. Player actions such as footsteps, door opens, and knocked-over objects would emit positioned events with a volume scalar, and GuardSensor would aggregate received events within a radius rather than checking distance to the player transform each frame. This would make hearing directional (the guard could face toward a sound source rather than just toward the player), would scale naturally to multiple noise sources, and would let me give the player real audio affordances such as running on gravel versus carpet, or opening a heavy door versus a curtain. The trade-off is that it pushes complexity from the sensor into every player-side action that produces a sound, and it introduces a new system that needs its own debug visualisation. For a single-guard encounter, the current model is sufficient. For a multi-guard level or any future extension, the noise-event system is the right place to invest.

The insight that refined my thinking is the difference between Unity's hand-roll-it culture and Unreal's bundle-it-in culture. Writing LeafSearch as a hand-rolled NavMesh-sampling routine taught me what an EQS query actually does under the hood. If I had reached for EQS in Unreal first, the cover-finding behaviour would have been one line of configuration and I would have learned nothing about why it works. The engine you pick shapes not just what you ship but what you understand. For my next project I would still pick Unity but specifically because the gap forces the learning, not despite it.

Frank self-assessment:

The CA2 marker awarded 91/100 and the feedback identified two specific issues, the CooldownDecorator behaviour on a child that returns Success immediately and the instance-state in LeafSearch not resetting on re-entry that I had not noticed myself. Both are addressed in CA3 but the more useful thing I took away from CA2 was that the BT framework I built was correct on the happy path and brittle at the boundaries. CA3 was the discipline of finding the boundaries. The LKP-off-NavMesh bug, the path-resolution-latency bug, the decorator wrap order bug, the GC allocations in the cone renderer: none of these were design failures, they were assumptions I had made without testing. The habit I want to carry into professional work is profiling early and writing tests for the unhappy paths first. Single-player single frame testing creates false confidence in real-time systems, and the CA2 marker's feedback is the most concrete evidence I have of that.

Appendix A: File inventory and milestones

Original work for CA3:

Assets/Scripts/CA3_AI/ contains the new code added for CA3: Blackboard.cs, BTDecorator.cs, DebugOverlay.cs, GuardBT.cs, GuardMovement.cs, GuardSensor.cs, GuardVisionCone.cs, LeafChase.cs, LeafInvestigate.cs, and LeafSearch.cs.

Carried forward from CA2:

Assets/Scripts/CA2/ contains the unchanged CA2 files: BTNode.cs, BTComposite.cs, LeafPatrol.cs, GuardAI.cs, and SuspicionSystem.cs. These were left untouched so the marker can verify which work belongs to which submission.

Carried forward from CA1:

Assets/Scripts/CA1_AI/ contains the original CA1 navigation and gameplay scripts: GameUI.cs, GoalZone.cs, SlidingDoor.cs, and AI/GuardCatch.cs. The legacy CA1 GuardMovement.cs is preserved alongside the CA3 movement, which lives in CA3_AI/.

Evidence:

Docs/Screenshots/CA3_AI_Screenshots/ contains the four state captures.

Docs/ProfilerCaptures/AI_Profiler/ contains the three profiler captures (Before, After, Fixed), with interpretation in Docs/CA3_ProfilingPack/ProfilingNotes.md. CA2 carry-forward material is at Docs/CA2_TechNote.md, Docs/CA2_CriticalDiscussion.md, and Docs/CA2_Evidencelog.md. The CA1 technical note is at Docs/TechNote.md, the early planning document is at Docs/CA3_Plan.md, and Docs/README_Evidence.md is the evidence index.

Git milestone tags on feature/ca3-ai:

ca3-start marks the first vertical slice scene. ca3-alpha-ai marks the CA2 marker fixes plus the sight confidence layer landing. ca3-beta-ai marks velocity-aware hearing and visualisation integrated and debuggable. ca3-profiling-pack-ai marks the cached buffer optimisation and profiler captures being committed. ca3-submit-ai is the final submission commit. The -ai suffix on alpha/beta/profiling-pack/submit exists to disambiguate from the parallel feature/ca3-network branch's CA3 GameDev tags, since both submissions share this repository. The ca3-start tag is shared because it predates the branch split.

Attribution and AI Use Statement:

Third-party packages used (all by Unity Technologies, all installed via Package Manager): AI Navigation (com.unity.ai.navigation) for guard pathfinding, TextMeshPro (com.unity.textmeshpro) for debug overlay text, Input System (com.unity.inputsystem) for player input and the F1 overlay toggle, and Universal Render Pipeline (com.unity.render-pipelines.universal) for the cinematic lighting and post-processing pass. Module-provided assets were used as supplied with no modifications.

AI assistance: Claude (Anthropic) was used during development for debugging assistance and research support, particularly around profiler interpretation and the engine-comparison and serious-games discussion in Section 3. All implementation decisions, written work, and submitted code are my own. The AI was used as a sounding board and research aid with some contribution towards the code primarily for debugging.